

Group members: Ashwin Varkey 23115420

Chukwudi Williams 23061170

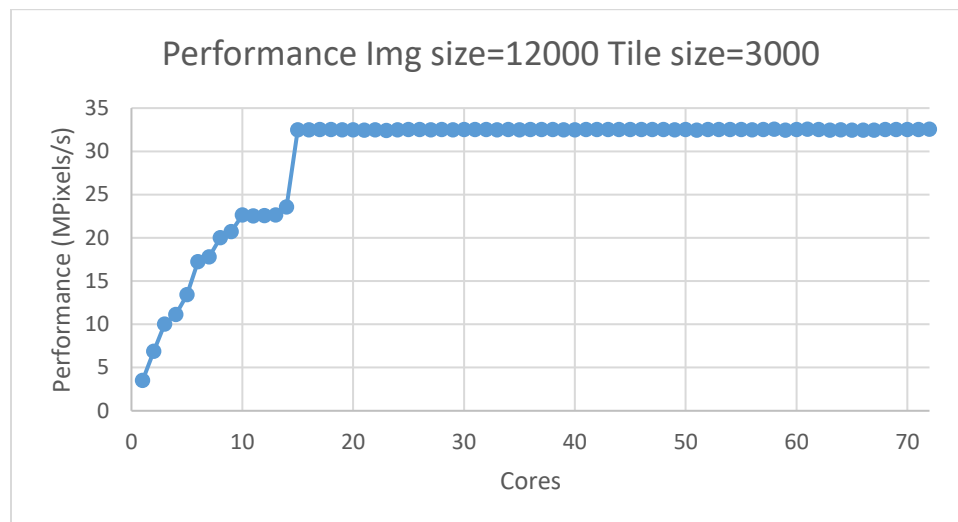
Manik Malik 23169717

Assignment 9

1. Parallel ray tracer.

(a) `icx -qopenmp -Ofast -xHost -fno-alias test.cpp -o a.out`

`sruntime --cpu-freq=2400000-2400000 ./a.out`

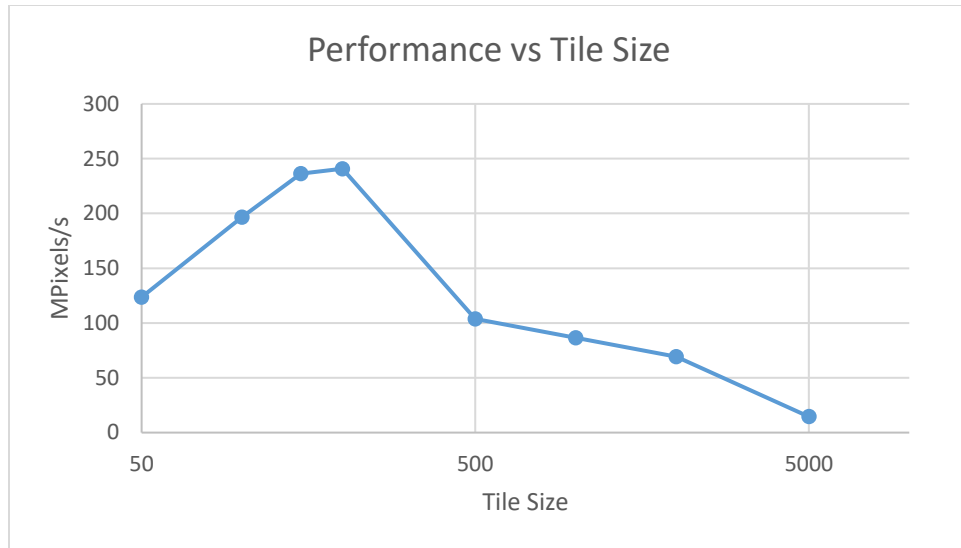


(b) Barrier synchronization can be a potential problem if the workload distribution is uneven or if some threads finish their work earlier than others. **For tile size of 3000, we are getting the stalling of performance at around 15 cores upto 72 cores (Performance: 32 MPixels/s) and the code does not achieve a linear speedup with 72 cores.** Some possible strategies include:

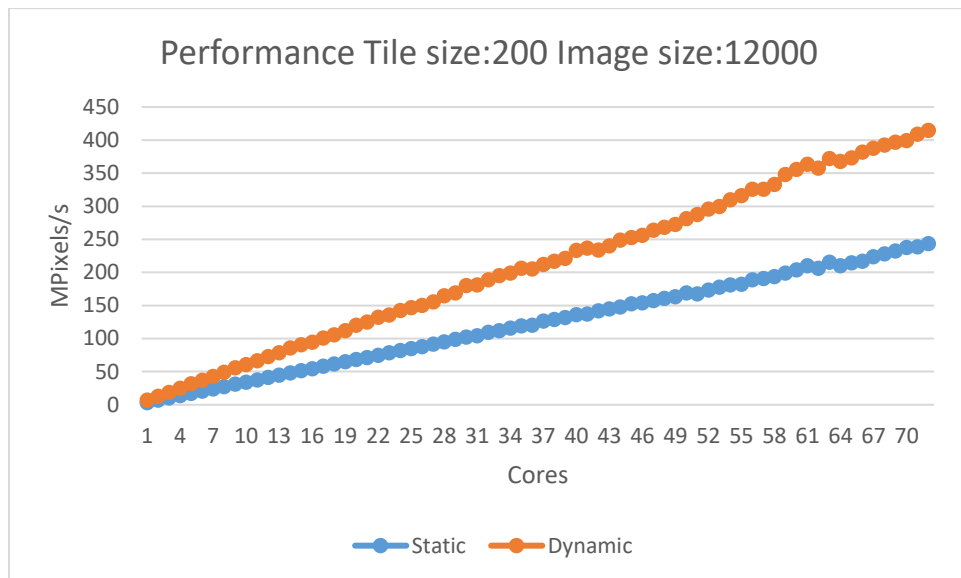
- Load balancing: Ensure that the workload is evenly distributed among the threads by dynamically assigning tiles based on the current workload status.
- Fine-grained parallelization: Instead of parallelizing at the tile level, we can explore parallelizing at a finer granularity, such as dividing each tile into smaller blocks and assigning those to threads.

(c) Since the code is not running 72 times faster than sequential mode, we have improved the scaling using the following strategies:

- Privatization of 'tile' to get proper image after parallelization.
- Fix Optimal Tile size from the graph below which is a max performance around 240 MPixels/s at **200 Tile Size**
- Parallelization of two for loops inside main function and specifies how many loops in a nested loop should be collapsed into one large iteration space and divided according to the schedule clause. The sequential execution of the iterations in all associated loops determines the order of the iterations in the collapsed iteration space.



Now the code scales linearly with respect to the number of cores.



Code:

```
int main(int argc, char** argv)
{
    int size = 12000, tileSize = 3000, xtiles, ytiles, xc, yc, count, tilebase, i;
    char* tile, * picture;
    double ws, we, cs, ce;

    // Run the loop from 1 to 72 threads

    for (int num_threads = 1; num_threads <= 72; num_threads++)
    {
        omp_set_num_threads(num_threads); // Set the number of threads
    }
}
```

```

#pragma omp parallel private(tile)
{
    tile = (char*)malloc(tilesz * tilesz * sizeof(char));
    if (tile == NULL)
    {
        fprintf(stderr, "Could not allocate tile memory!\n");
        exit(1);
    }

    if ((picture = (char*)malloc(size * size * sizeof(char))) == NULL)
    {
        fprintf(stderr, "Could not allocate picture memory!\n");
        exit(1);
    }

    for (i = 0; i < size * size * sizeof(char); ++i)
        picture[i] = 0;

    ws = getTimeStamp();

    /* number of tiles in x and y direction */
    xtiles = size / tilesz;
    ytiles = size / tilesz;

    count = 0;
#pragma omp for private(xc,i) schedule(dynamic) collapse(2)
    for (yc = 0; yc < xtiles; yc++)
    {
        for (xc = 0; xc < ytiles; xc++)
        {
            /* calc one tile */
            calc_tile(size, xc * tilesz, yc * tilesz, tilesz, tile);

            /* copy to picture buffer */
            for (i = 0; i < tilesz; i++)
            {
                tilebase = yc * tilesz * tilesz * xtiles + xc *
tilesz;
                memcpy((void*)(picture + tilebase + i * tilesz * xtiles),
                    (void*)(tile + i * tilesz),
                    tilesz * sizeof(char));
            }
            count++;
        }
    }

    we = getTimeStamp();
    double perf = (size * size) / ((we - ws) * 1e6);
    fprintf(stderr, "Threads: %d\t Time: %f\t Perf: %f\n", num_threads, we - ws,
perf);

    free(tile);
    free(picture);
}

exit(0);
return 0;
}

```

2. π by the Monte Carlo method. The quarter unit circle in the first quadrant with origin at (0,0) and radius 1 has an area of $\pi/4$. Look at random number pairs in $[0,1] \times [0,1]$.

We use `rand_r` because it is thread safe in each of the parallel thread. **Each thread generates its own separate random number sequence using the `rand_r()` function, ensuring that different threads don't interfere with each other's random numbers.**

Report the relative error you can achieve with 36 threads on one Fritz socket and 5×10^9 points

Relative error = 4.2512×10^{-6}

Performance = 2358451526.950131

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>

double getTimeStamp()
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1.e-9;
}

int main() {
    double wct_start, wct_end;
    unsigned int i, sum = 0, N = 3 * pow(10, 9);
    double x, y, a = 10;

    srand(time(NULL));

#pragma omp parallel private(x, y) reduction(+:sum)
    {
        unsigned int seed = (unsigned int)time(NULL) ^ omp_get_thread_num();

#pragma omp for
        for (i = 0; i < N; i++) {
            if (a > 10) {
                printf("Test Loop");
            }
        }

        wct_start = getTimeStamp();

#pragma omp for
        for (i = 0; i < N; ++i) {
            x = (double)rand_r(&seed) / RAND_MAX;
            y = (double)rand_r(&seed) / RAND_MAX;

            if ((x * x + y * y) < 1) {
                ++sum;
            }
        }
    }
}
```

```

    wct_end = getTimeStamp();
}

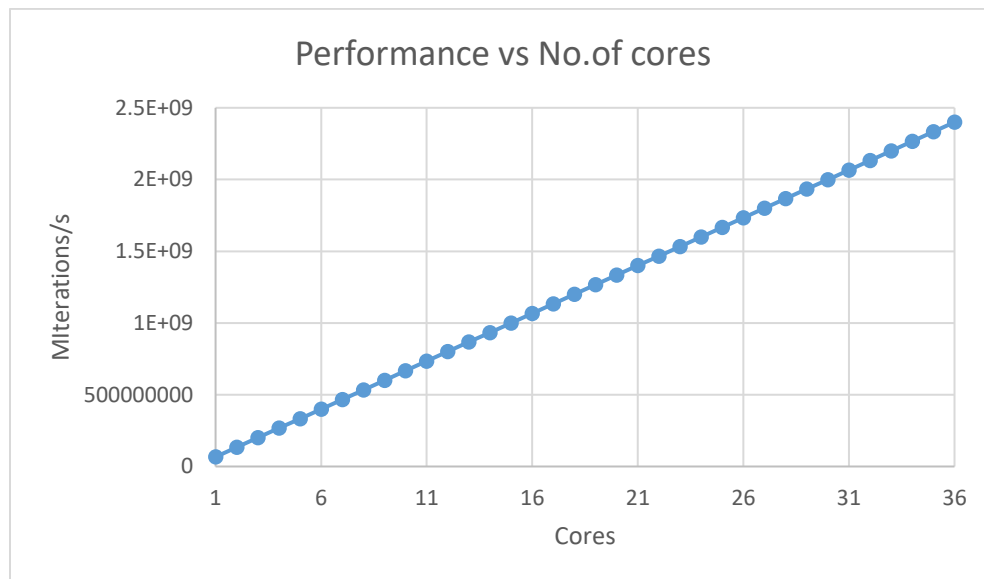
double perf = (double)N / (double)(wct_end - wct_start);
double pi = 4 * ((double)(sum) / (double)N);

printf("Relative error = %.6f\n", (M_PI - pi) / M_PI);
printf("Performance = %.6f\n", perf);

return 0;
}

```

Initially it doesn't scale due to overheads. After Bogus parallel region is added before actual benchmark loop, the overhead is reduced



Yes, the performance of the code scales with increase in number of cores.